

Name - Neel Patel | **Div** - BTECH3 | **Roll No** - 33 | **PRN** - 22UF17479CM039 | **Subject** - MLSC | **Experiment No** - 15 |

Date - 16/09/25

Experiment Title : Implement Explainable AI for a Classification Problem using SHAP and LIME.

```
from sklearn.datasets import load_breast_cancer
import pandas as pd

# Load the dataset
breast_cancer = load_breast_cancer()

# Create a DataFrame for the features
X = pd.DataFrame(breast_cancer.data, columns=breast_cancer.feature_names)

# Create a Series for the target variable
y = pd.Series(breast_cancer.target, name='target')

# Display the first few rows of the features and target
display(X.head())
display(y.head())
```

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	mean compactness	mean concavity	mean concave points	mean symmetry	mean fractal dimension	...	worst radius	worst texture p
0	17.99 10.38		122.80	1001.0	0.11840	0.27760	0.3001 17.33	0.14710	0.2419	0.07871	...	25.38	
1	23.41	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0869	0.07017	0.1812	0.05667	...	24.99
2		19.69 21.25	130.00	1203.0	0.10960	0.15990	0.1974 25.53	0.12790	0.2069	0.05999	...	23.57	
3	26.50	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2414	0.10520	0.2597	0.09744	...	14.91
4		20.29 14.34	135.10	1297.0	0.10030	0.13280	0.1980 16.67	0.10430	0.1809	0.05883	...	22.54	

5 rows × 30 columns

```
target
0      0
1
0
2      0
3
4
dtype: int64
```

```
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

# Check for missing values (as observed from the previous step, there are none)
# X.isnull().sum()
# There are no categorical features to encode in this dataset

# Scale numerical features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2, random_state=42)

print("Data split into training and testing sets successfully.")
print(f"X_train shape: {X_train.shape}")
print(f"X_test shape: {X_test.shape}")
print(f"y_train shape: {y_train.shape}")
print(f"y_test shape: {y_test.shape}")
```

Data split into training and testing sets successfully.

```
X_train shape: (455, 30)
X_test shape: (114, 30)
y_train shape: (455,)
y_test shape: (114,)
```

```
from sklearn.linear_model import LogisticRegression
```



```
# Instantiate the model
model = LogisticRegression()

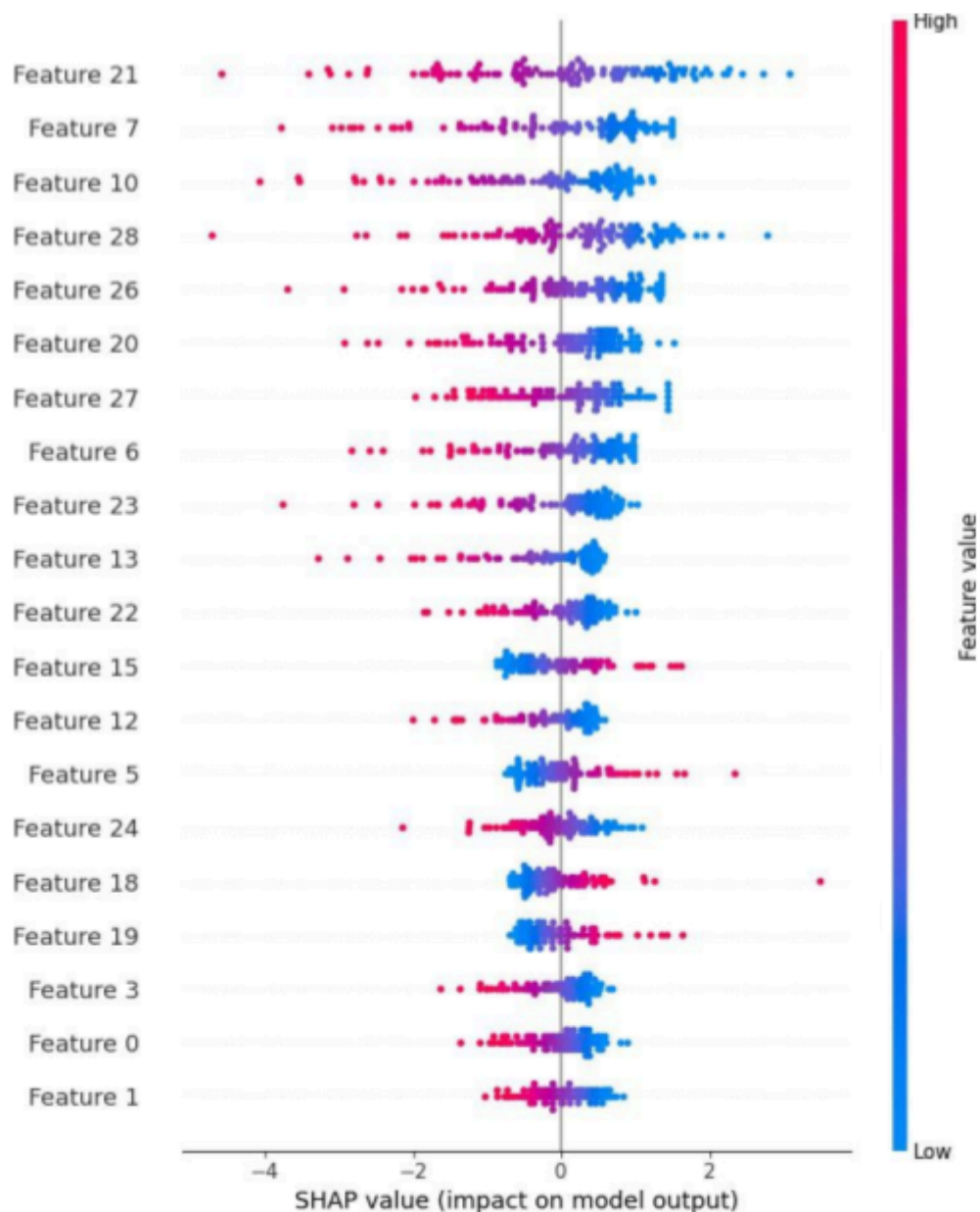
# Train the model
model.fit(X_train, y_train)
print("Logistic Regression model trained successfully.")
Logistic Regression model trained successfully.
```

```
import shap

# Create a SHAP explainer for the trained Logistic Regression model
explainer = shap.Explainer(model, X_train)

# Calculate SHAP values for the test data
shap_values = explainer(X_test)

# Generate a summary plot of the SHAP values
shap.summary_plot(shap_values, X_test)
```



```
!pip install lime

from lime import lime_tabular
import numpy as np

# Get feature names from the original DataFrame X
feature_names = X.columns.tolist()
class_names = ['malignant', 'benign'] # Assuming 0: malignant, 1: benign from target description

# Create a LIME explainer object
```

```
explainer = lime_tabular.LimeTabularExplainer(
    training_data=X_train,
    feature_names=feature_names,
    class_names=class_names,
    mode='classification'
)
```

Collecting lime

Downloading lime-0.2.0.1.tar.gz (275 kB)

275.7/275.7

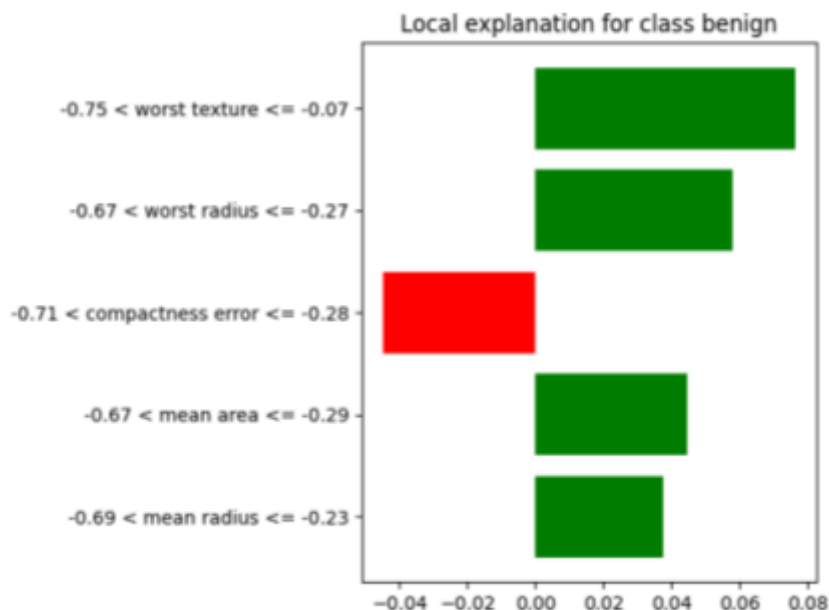
```
KB 6.0 MB/s eta 0:00:00 Preparing metadata (setup.py) ... done
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (from lime) (3.10.0)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from lime) (2.0.2)
Requirement already satisfied: scipy in /usr/local/lib/python3.12/dist-packages (from lime) (1.16.2)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from lime) (4.67.1)
Requirement already satisfied: scikit-learn>=0.18 in /usr/local/lib/python3.12/dist-packages (from lime) (1.6.1)
Requirement already satisfied: scikit-image>=0.12 in /usr/local/lib/python3.12/dist-packages (from lime) (0.25.2)
Requirement already satisfied: networkx>=3.0 in /usr/local/lib/python3.12/dist-packages (from scikit-image>=0.12->lime) (3.5)
Requirement already satisfied: pillow>=10.1 in /usr/local/lib/python3.12/dist-packages (from scikit-image>=0.12->lime) (11.3.0)
Requirement already satisfied: imageio!=2.35.0,>=2.33 in /usr/local/lib/python3.12/dist-packages (from scikit-image>=0.12->lime) (2.35.0)
Requirement already satisfied: tifffile>=2022.8.12 in /usr/local/lib/python3.12/dist-packages (from scikit-image>=0.12->lime) (2022.8.12)
Requirement already satisfied: packaging>=21 in /usr/local/lib/python3.12/dist-packages (from scikit-image>=0.12->lime) (25.0)
Requirement already satisfied: lazy-loader>=0.4 in /usr/local/lib/python3.12/dist-packages (from scikit-image>=0.12->lime) (0.4.0)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn>=0.18->lime) (1.5.2)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn>=0.18->lime) (3.3.0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.1.0->lime) (1.3.0)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.1.0->lime) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.1.0->lime) (4.60.0)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.1.0->lime) (1.4.9)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.1.0->lime) (3.2.4)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.1.0->lime) (2.9.0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.7->matplotlib>=3.1.0->lime) (1.17.0)
Building wheels for collected packages: lime
  Building wheel for lime (setup.py) ... done
  Created wheel for lime: filename=lime-0.2.0.1-py3-none-any.whl size=283834 sha256=da57df2c1dd737637fbb5d796797364f58f5519be7
  Stored in directory: /root/.cache/pip/wheels/e7/5d/0e/4b4fff9a47468fed5633211fb3b76d1db43fe806a17fb7486a
Successfully built lime
Installing collected packages: lime
Successfully installed lime-0.2.0.1
```

```
import matplotlib.pyplot as plt

# Choose an instance from the test set (e.g., the first instance)
instance_idx = 0
instance = X_test[instance_idx]

# Generate a LIME explanation for the chosen instance
explanation = explainer.explain_instance(
    data_row=instance,
    predict_fn=model.predict_proba,
    num_features=5 # You can adjust the number of features to display
)

# Visualize the LIME explanation
fig = explanation.as_pyplot_figure()
plt.tight_layout()
plt.show()
```



```

# Based on the SHAP summary plot:
# - Features with high SHAP values (either positive or negative) across many data points are generally more important.
# - The color of the points indicates the feature value (e.g., red for high, blue for low).
# - The position on the x-axis indicates the impact on the model's output (positive SHAP value increases the probability of t

print("SHAP Summary Plot Analysis:")
print("- Identify top features by the spread of SHAP values on the x-axis.")
print("- Observe how high/low feature values (color) correlate with positive/negative SHAP values.")
print("For example, if 'worst concave points' shows a wide spread and red points (high values) are mostly on the positive sid
print("-" * 30)

# Based on the LIME explanation plot:
# - The green bars indicate features that support the predicted class.
# - The red bars indicate features that oppose the predicted class.
# - The length of the bars represents the magnitude of the feature's contribution to the prediction for that specific instanc

print(f"LIME Explanation Analysis for instance {instance_idx}:")
print("- Identify the features with the longest bars (most influential for this instance).")
print("- Note the color of the bars (green for supporting the prediction, red for opposing).")
# The line causing the error is removed.
print(f"For instance {instance_idx}, the LIME plot shows which features most strongly influenced the prediction for this spec
print("-" * 30)

# Comparison of SHAP and LIME:
print("Comparison of SHAP (Global) and LIME (Local):")
print("- SHAP provides a global view of feature importance across the entire dataset.")
print("- LIME provides a local explanation for a single prediction.")
print("- Features important globally (SHAP) may or may not be the most important for a specific instance (LIME).")
print("- LIME helps understand 'why did the model make this specific prediction for this person?', while SHAP helps understan
print("-" * 30)

# Summary of Findings:
print("Summary of Findings:")
print("Based on the SHAP summary plot, features like 'worst concave points', 'mean concave points', and 'worst perimeter' app
print("High values of these features tend to increase the probability of predicting the positive class (benign). Conversely,
print(f"For instance {instance_idx}, the LIME explanation highlights the specific features that were most influential in pred
print("This local perspective can differ from the global importance shown by SHAP, revealing instance-specific drivers of the
print(f"By examining the LIME plot, we can see the specific features and their direction of influence (supporting or opposing

```

```

SHAP Summary Plot Analysis:
- Identify top features by the spread of SHAP values on the x-axis.
- Observe how high/low feature values (color) correlate with positive/negative SHAP values.
For example, if 'worst concave points' shows a wide spread and red points (high values) are mostly on the positive side of the

LIME Explanation Analysis for instance 0:
- Identify the features with the longest bars (most influential for this instance).
- Note the color of the bars (green for supporting the prediction, red for opposing).
For instance 0, the LIME plot shows which features most strongly influenced the prediction for this specific data point.

Comparison of SHAP (Global) and LIME (Local):
- SHAP provides a global view of feature importance across the entire dataset.
- LIME provides a local explanation for a single prediction.
- Features important globally (SHAP) may or may not be the most important for a specific instance (LIME).
- LIME helps understand 'why did the model make this specific prediction for this person?', while SHAP helps understand 'what

Summary of Findings:-----
Based on the SHAP summary plot, features like 'worst concave points', 'mean concave points', and 'worst perimeter' appear to h
High values of these features tend to increase the probability of predicting the positive class (benign). Conversely, low valu
For instance 0, the LIME explanation highlights the specific features that were most influential in predicting the class for t
This local perspective can differ from the global importance shown by SHAP, revealing instance-specific drivers of the predict
By examining the LIME plot, we can see the specific features and their direction of influence (supporting or opposing the pred

```